

HACKATHON: AI-ASSISTED ONBOARDING in Electrical Engineering

ROHDE & SCHWARZ
Make ideas real



KNUST
KWAME NKRUMAH UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Ready to build AI solutions that make lab onboarding faster,
safer and more effective?

Overview challenge tracks

- **Augmented Reality Design:** Create interactive, 3D visualizations of instrument parts, connectors, and front-panel overlays using augmented reality technology.
- **Virtual Lab Assistant:** A conversational AI lab assistant that guides users through step-by-step procedures, safety checks, and calibration tasks with voice and/or chat interactions.
- **Intelligent Instrumentation Workflow Builder:** Automatic workflow generator that uses intent-based inputs to configure instrument settings and create customized measurement scripts, streamlining experimental workflows.
- **AI-Powered Troubleshooting Tool:** A troubleshooting assistant that leverages waveform/trace analysis and anomaly detection algorithms to identify issues and provide expert recommendation for resolutions.
- **Personalized Onboarding Experience:** An adaptive competency tracking system that uses analytics and machine learning to tailor onboarding content, ensuring new users quickly develop essential skills and expertise.

Challenge track 1 – Augmented Reality (AR) Design

Item	details
Problem statement	New students struggle to identify instrument controls, connectors and front-panel functions quickly. Create an AR solution that overlays contextual, interactive guidance on real instruments (or their images) to accelerate recognition, correct cabling, knob/switch operation, and safety checks.
Objectives	Provide real-time visual overlays identifying parts and safe operation zones. Offer step-by-step contextual tasks (e.g., “connect probe to CH1, set coupling to DC, set timebase to X”). Reduce time-to-first-measurement and error rates for novice users.
User stories	“As a first-year student, I want to point my phone at an oscilloscope and see labels on ports and knobs so I can plug probes correctly.” “As a lab assistant, I want students to follow highlighted safety zones to avoid hazardous connections.”
Required inputs / datasets	High-resolution images of instrument panels and connectors (multiple angles). 3D models or exploded views if available (R&S assets ideal). Mapping of front-panel controls to instrument API commands (control matrix). Example workflows (step sequences) for common tasks.
Hardware & APIs available	R&S scopes, signal generators R&S instrument control APIs/drivers for querying state and applying settings Mobile phones/tablets (teams supply) optional - AR headsets if available. optional - Lab cameras for multi-view recognition
tech stack	ARKit/ARCore, Unity or native mobile frameworks, OpenCV for markerless tracking, Mediapipe for hand/pose detection, WebAR for browser demos. Backend: small REST service to translate overlays to instrument API calls (Python/Flask).

	Constraints & performance targets Overlay registration error < 2–3 cm for key connectors; recognition time < 1s on target device. System must not send unsafe instrument commands without explicit confirmation.
Deliverables & demo criteria	Live demo: point device at instrument and show overlays + at least one guided workflow that results in a successful measurement as verified by the instrument API. Code + model assets, README with calibration steps, instructions for using on alternate instruments.
Evaluation metrics	Usability: time saved on identification / task completion (measured in user test). Accuracy: correct identification rate of components. Robustness: works under different lighting/angles. Safety: prevents or flags hazardous configurations.
Safety & lab considerations	AR must not automatically change power/voltage settings without explicit user confirmation and a safety interlock. Encourage simulated mode for first demos where hardware control is risky
Starter ideas & stretch goals	Starter: image-based labels and hotspot overlays for a single instrument model. Stretch: dynamic overlays showing measurement readings, live pointer to connectors, voice prompts, multi-instrument AR workflows.
Mentor prompts	How to calibrate overlay to different instrument models? What safety checks to add before an action?

Challenge track 2 – Virtual Lab Assistant (Conversational / Voice)

Item	details
Problem statement	Novices lack accessible stepwise guidance and have trouble translating lab procedures into commands. Build a conversational AI (voice and/or chat) that guides users through setup, checks safety, walks through calibration and measurement steps, and optionally triggers instrument APIs under controlled conditions.
Objectives	Provide natural language guidance (text/voice) for tasks like “measure a 1 kHz sinus with CH1”. Validate user inputs and confirm safety checks before permitting risky actions. Integrate with instrument APIs to read state, provide feedback, and optionally push configuration commands with explicit confirmation.
User stories	“As a student, I want to ask ‘How do I measure a 1 kHz sine?’ and receive a step-by-step plan.” “As an instructor, I want to see a log of student interactions and common failure points.”
Required inputs / datasets	FAQ & procedure documents for basic tasks. Example transcripts of guidance sessions (if available). Instrument state schemas and command sets. Optional: annotated troubleshooting dialogs.
Hardware & APIs available	R&S instruments via API for state/readback. Microphone/headset for voice; teams supply hardware. Cloud speech-to-text / TTS (or local open-source stack).
tech stack	Dialog management: Rasa, Microsoft Bot Framework, or LLM-based agents (OpenAI/LLMs if allowed) with retrieval augmented generation (RAG) over procedural docs. Speech: Vosk (local) or cloud STT/TTS. Backend: Python with instrument API integration; conversational state store.
Constraints & performance targets	Response latency ideally < 2s for local queries; confirmations required for any state-changing operation.

	Privacy: do not log PII without consent.
Deliverables & demo criteria	Live demo showing at least two workflows (one simple, one safety-critical) where the assistant guides a user and optionally performs an instrument configuration after confirmation. Conversation transcripts, code repo, README on how to connect to instruments or simulator.
Evaluation metrics	Helpfulness: user task success rate and satisfaction scores. Accuracy: correctness of instructions and safety checks. Robustness: handling ambiguous queries & error recovery
Safety & lab considerations	The assistant must always require an explicit confirmation step before sending instrument-changing commands and should include safety checks (voltage thresholds, grounding instructions). Use simulated instruments for demos that would otherwise risk hardware damage.
Starter ideas & stretch goals	Starter: chat UI that returns step sequences from a recipe database. Stretch: integrate voice, auto-configure instruments after user confirmation, adapt explanations to skill level.
Mentor prompts	What are the minimum safety confirmations required? Which commands should be blocked unless an instructor approves?

Challenge track 3 – Intelligent Instrumentation Workflow Builder

Item	details
Problem statement	Configuring a measurement chain and translating intent into instrument settings is time-consuming. Build a tool that accepts a high-level intent (e.g., “measure SNR of amplifier at 1 GHz”) and auto-generates a multi-instrument workflow script, instrument settings, and a reproducible measurement procedure.
Objectives	Translate natural language or structured intent into sequenced instrument commands and test scripts. Validate generated workflows against device/measurement constraints (frequency limits, input ranges). Output runnable scripts (Python/SCPI) or instrument API calls plus a human-readable checklist.
User stories	“As a student, I want to describe a measurement in plain language and get a script that configures the instruments and logs results.”
Required inputs / datasets	Catalog of instrument capabilities (ranges, commands). Example measurement recipes and expected outputs. Safety rules and instrument limits.
Hardware & APIs available	R&S instruments with programmable APIs and SCPI support; simulated drivers for early testing.
tech stack	Intent parsing: LLMs or rule-based parser + slot filling. Workflow synthesis: templating engine to emit Python/SCPI scripts. Validation engine: rule checker for constraints and safety.
Constraints & performance targets	Generated workflows should run in simulator mode without error; real-hardware runs must require explicit human approval. Script generation time < 30s for simple requests.
Deliverables & demo criteria	Demo of at least two intent→script flows, showing validation, simulated run, and (optionally) safe real hardware run. Provide generated scripts, README, and test dataset showing expected measurement outputs.
Evaluation metrics	Correctness: scripts produce expected configuration and measurement in simulation.

	Coverage: can handle varied intents and instruments. Safety: prevents commands that violate limits or risk damage. Readability: generated human checklist clarity.
Safety & lab considerations	Include pre-run safety checklist; block commands above safe thresholds. Require “confirm & run” verb before real hardware execution.
Starter ideas & stretch goals	Starter: map a fixed set of intents to script templates for oscilloscope and signal generator. Stretch: fully natural-language parsing and multi-instrument synchronization scripts.
Mentor prompts	How to model device limitations? How to create safe defaults for unknown parameters?

Challenge track 4 – AI-Powered Troubleshooting Tool

Item	details
Problem statement	When measurements look wrong or circuits fail, novices don't know how to diagnose root causes. Develop an AI tool that analyzes measurement traces/logs and suggests probable faults and step-by-step remediation actions.
Objectives	Automatically classify anomalies from oscilloscope traces, spectra, or network analyzer responses and propose prioritized fixes. Provide explainable diagnostics (why suggestion is made) and actionable next steps.
User stories	"As a student, I run a measurement and see a strange waveform. I want an assistant to tell me likely causes—bad grounding, probe setup, faulty component—and how to check."
Required inputs / datasets	Labeled datasets of waveforms/traces with annotated faults (probe mis-connection, ground loop, saturation, clipping, noise, cable fault). Spectral traces with typical interference signatures, S-parameter anomalies, power supply logs. Troubleshooting guides and mapping to corrective actions.
Hardware & APIs available	R&S instrument capture via API for live traces; simulated traces if dataset gaps exist. Image capture for physical faults (loose connectors, visible damage) - optional.
tech stack	Signal processing: SciPy, NumPy for feature extraction; Wavelet transforms; FFT. ML: classification models (CNNs on spectrograms, time-series networks, anomaly detectors). Explainability: SHAP/LIME or rule-based explanations
Constraints & performance targets	Classification accuracy target > 80% on key fault classes (depending on dataset). Latency: results within 5–10s for interactive use.
Deliverables & demo criteria	Demonstration: feed in 3–5 real or simulated faulty traces and show diagnosis plus prioritized remediation steps; show one case where the tool prevents unsafe action (e.g., warns about overvoltage). Repo, model weights (or training recipe), evaluation on validation set.
Evaluation metrics	Diagnostic precision & recall per fault class. Practicality: relevance and actionability of suggested steps (judged by mentors).

	Explainability: clarity of why a diagnosis was made.
Safety & lab considerations	Tool must not recommend unsafe fixes (e.g., bypass safety interlocks); include safety warnings. Emphasize using simulated traces for testing destructive faults.
Starter ideas & stretch goals	Starter: simple classifier for probe/mis-duration/noise vs. correct waveform. Stretch: cross-instrument correlation (spectrum + time domain), visual inspection integration.
Mentor prompts	Which fault classes are most common in your lab? Can mentors help curate labeled examples?

Challenge track 5 – Personalized Onboarding Experience (Adaptive Learning)

Item	details
Problem statement	One-size-fits-all onboarding wastes time. Create an adaptive learning system that assesses a newcomer's current skills and dynamically presents targeted tutorials, exercises and stepwise instrument tasks to maximize learning efficiency.
Objectives	Deliver an initial competency check and then adapt content sequencing based on performance. Track progress, recommend next exercises, and provide instructors with analytics on cohort skill gaps.
User stories	"As a new lab user, I want personalized exercises that focus on my weak points (e.g., probe compensation, trigger settings)." "As an instructor, I want a dashboard showing class mastery levels and recommended remediation sessions."
Required inputs / datasets	Curriculum modules and assessment items (quizzes, practical tasks). Performance benchmarks and expected completion times for tasks. Examples of student interactions (if available) or simulated assessment results.
Hardware & APIs available	Access to R&S simulators and instrument APIs for automatic grading of practical tasks. Web platform for quizzes and progress tracking
tech stack	- Backend: Python with a rules engine for adaptivity or an RL-based curriculum selector for advanced teams. Frontend: responsive web app (React/Vue) or mobile UI. Analytics: simple learner model (e.g., Elo or Item Response Theory) to estimate competency levels.
Constraints & performance targets	Initial assessment should be < 20 minutes and place user into an appropriate learning path. System should recommend next task within < 2s after assessment submission
Deliverables & demo criteria	Demo: run a sample learner through assessment → adaptive path → practical simulator task graded automatically. Provide learner model, sample content, instructor dashboard mockups, repo and README.

Evaluation metrics	Learning efficiency: reduction in time to competency (simulated or pilot). Adaptivity quality: correctness of recommended next steps vs. expert choices. Usability: learner satisfaction and clarity of feedback.
Safety & lab considerations	Ensure graded practical tasks default to simulator mode until safety induction is completed. Include checks that block on-hardware tasks for unqualified users.
Starter ideas & stretch goals	Starter: rule-based adaptive flow with few modules (basic, intermediate). Stretch: use ML to predict most effective next exercise for each learner and integrate with AR/assistant for guided practice.
Mentor prompts	What are key competencies for first-year EE lab onboarding? Can KNUST provide baseline assessment items?